JAVA LINKED LIST
COMPLETE INTERVIEW HANDBOOK

Singly • Doubly • Circular • Pointer Patterns • Interview Algorithms

Detailed concepts, rules, formulas, pointer diagrams, dry-run logic, full Java implementations, complexity and problem bank

Java | IntelliJ IDEA | Beginner to Advanced

How to use this handbook

This guide is intentionally detailed. Each code listing is a complete Java class; save it in a file with the same name as its public class. Run one class at a time in IntelliJ IDEA. Most examples use a nested Node class so they compile independently.

Standard import: `import java.util.*;` Add it when using library classes such as NoSuchElementException, although fully qualified names are used in some listings.

Interview habit: state the node structure, draw the links, identify edge cases, explain pointer updates, then give time and auxiliary-space complexity.

1. Linked List: definition and mental model

A linked list is a sequence of nodes. A node stores a value and a reference to another node. Unlike an array, nodes are not required to be stored in adjacent memory locations.

```
head
 |
 v
[10 | next] -> [20 | next] -> [30 | null]
```

Singly linked node: value + next. **Doubly linked node:** value + prev + next. **Circular list:** the final link returns to the first node.

Why it matters: access by index is O(n), because links must be followed. Insertion/deletion is O(1) only when the relevant node or predecessor is already known. Finding that position may itself cost O(n).

2. Types of linked lists

2.1 Singly linked list

```
head -> [A|next] -> [B|next] -> [C|null]
```

Only forward traversal. A node does not know its predecessor. The last node's next is null.

2.2 Doubly linked list

```
null <- [prev|A|next] <-> [prev|B|next] <-> [prev|C|next] -> null
```

Each node points forward and backward. Deleting a known node is O(1), since both neighbors are available. More memory is used for prev.

2.3 Circular singly linked list

```
head -> [A] -> [B] -> [C]
         ^             |
         |_____________|
```

tail.next == head. There is no null terminator. Use a do-while traversal or explicitly stop when the pointer returns to head.

2.4 Circular doubly linked list

```
head <-> [A] <-> [B] <-> [C] <-> head
```

In a non-empty circular doubly linked list, head.prev is tail and tail.next is head.

3. Complexity comparison

Operation	ArrayList	Singly linked list
Get index i	O(1)	O(n)
Search	O(n)	O(n)
Insert at head	O(n) shifts	O(1)
Delete head	O(n) shifts	O(1)
Insert after known node	O(n) shifts	O(1)
Append	O(1) amortized	O(n), O(1) with tail
Extra link memory	No per-element next	O(n) references

Always qualify complexity: insertion after a known node is O(1), but searching for that node first is O(n).

4. Build a singly linked list from scratch

Maintain head, tail and size. Empty-list invariant: head == null and tail == null. Non-empty invariant: tail.next == null.

```
public class SinglyLinkedList {
    static class Node {
        int val;
        Node next;
        Node(int val) { this.val = val; }
    }

    private Node head, tail;
    private int size;

    public int size() { return size; }
    public boolean isEmpty() { return size == 0; }
```

```
    public void addFirst(int val) {
        Node node = new Node(val);
        node.next = head;
        head = node;
        if (tail == null) tail = node;
        size++;
    }

    public void addLast(int val) {
        Node node = new Node(val);
        if (head == null) head = tail = node;
        else {
            tail.next = node;
            tail = node;
        }
        size++;
    }

    public int removeFirst() {
        if (head == null) throw new java.util.NoSuchElementException();
        int ans = head.val;
        head = head.next;
        size--;
        if (head == null) tail = null;
        return ans;
    }

    public void display() {
        Node cur = head;
        while (cur != null) {
            System.out.print(cur.val + " -> ");
            cur = cur.next;
        }
        System.out.println("null");
    }

    public static void main(String[] args) {
        SinglyLinkedList list = new SinglyLinkedList();
        list.addFirst(20);
        list.addFirst(10);
        list.addLast(30);
        list.display(); // 10 -> 20 -> 30 -> null
        System.out.println(list.removeFirst()); // 10
        list.display(); // 20 -> 30 -> null
    }
}
```

addFirst O(1), addLast O(1) with tail, removeFirst O(1), display O(n).

5. Traverse, count and search

Traversal starts at head and follows next until null. Each node is visited once. Search returns the zero-based index or -1.

```
public class TraverseSearch {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static int count(Node head) {
        int n = 0;
        while (head != null) {
            n++;
            head = head.next;
        }
        return n;
    }

    static int search(Node head, int target) {
        int index = 0;
        while (head != null) {
            if (head.val == target) return index;
            head = head.next;
            index++;
        }
        return -1;
    }

    public static void main(String[] args) {
        Node h = new Node(5);
        h.next = new Node(9);
        h.next.next = new Node(13);
        System.out.println(count(h));       // 3
        System.out.println(search(h, 9));   // 1
        System.out.println(search(h, 99));  // -1
    }
}
```

Time O(n), extra space O(1).

6. Insert at a given index

Valid indices are 0 through n. For index 0, change head. Otherwise reach index-1, then splice: newNode.next = current.next; current.next = newNode.

```
public class InsertAtIndex {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node insert(Node head, int index, int value) {
        if (index < 0) throw new IndexOutOfBoundsException();
        Node node = new Node(value);
```

```
        if (index == 0) {
            node.next = head;
            return node;
        }

        Node cur = head;
        for (int i = 0; i < index - 1; i++) {
            if (cur == null) throw new IndexOutOfBoundsException();
            cur = cur.next;
        }
        if (cur == null) throw new IndexOutOfBoundsException();

        node.next = cur.next;
        cur.next = node;
        return head;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val + " "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h = new Node(10);
        h.next = new Node(30);
        h = insert(h, 1, 20);
        print(h); // 10 20 30
    }
}
```

Time O(n), extra space O(1), excluding the newly allocated node.

7. Delete at a given index

At index 0, return head.next. Otherwise reach the predecessor and bypass the target using cur.next = cur.next.next.

```
public class DeleteAtIndex {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node delete(Node head, int index) {
        if (index < 0 || head == null) throw new IndexOutOfBoundsException();
        if (index == 0) return head.next;

        Node cur = head;
        for (int i = 0; i < index - 1; i++) {
            if (cur.next == null) throw new IndexOutOfBoundsException();
            cur = cur.next;
        }
        if (cur.next == null) throw new IndexOutOfBoundsException();
        cur.next = cur.next.next;
        return head;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val + " "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h = new Node(1);
        h.next = new Node(2);
        h.next.next = new Node(3);
        h = delete(h, 1);
        print(h); // 1 3
    }
}
```

Time O(n). In a full list class, also update tail and size if the deleted node was the last node.

8. Reverse a singly linked list — most important

Rule: save the next node before reversing the current link. Set current.next = previous, then advance previous and current.

```
Before: null   1 -> 2 -> 3 -> null
               p    c
After:  null <- 1    2 -> 3 -> null
                p    c
Final:  null <- 1 <- 2 <- 3
```

```
public class ReverseLinkedList {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node reverse(Node head) {
        Node prev = null;
        Node cur = head;

        while (cur != null) {
            Node nextNode = cur.next; // save remainder
            cur.next = prev;          // flip this link
            prev = cur;               // move prev
            cur = nextNode;           // move current
        }
        return prev;
    }

    static void print(Node h) {
        while (h != null) {
            System.out.print(h.val + " ");
            h = h.next;
```

```
        }
        System.out.println();
    }

    public static void main(String[] args) {
        Node h = new Node(1);
        h.next = new Node(2);
        h.next.next = new Node(3);
        h = reverse(h);
        print(h); // 3 2 1
    }
}
```

Time: O(n). **Extra space:** O(1). Bug to avoid: if you overwrite cur.next before saving it, the remaining list is lost.

9. Recursive reversal

The recursive call reverses the suffix. On return, link the next node back to current, then set current.next to null to break the old forward edge.

```
public class RecursiveReverse {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node reverse(Node head) {
        if (head == null || head.next == null) return head;
        Node newHead = reverse(head.next);
        head.next.next = head;
        head.next = null;
        return newHead;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h = new Node(1);
        h.next = new Node(2);
        h.next.next = new Node(3);
        h = reverse(h);
        print(h); // 3 2 1
    }
}
```

Time O(n), recursion stack O(n). Iterative reversal is usually preferred when constant extra space is required.

10. Find middle — slow and fast pointers

Move slow one step and fast two steps. When fast reaches the end, slow is at the middle. For even length, this version returns the second middle.

```
public class MiddleOfLinkedList {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node middle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }

    public static void main(String[] args) {
        Node h = new Node(1);
        h.next = new Node(2);
        h.next.next = new Node(3);
        h.next.next.next = new Node(4);
        h.next.next.next.next = new Node(5);
        System.out.println(middle(h).val); // 3
    }
}
```

Time O(n), space O(1). To return the first middle for even length, initialize fast = head.next while slow = head.

11. Detect cycle — Floyd's algorithm

Slow moves one link; fast moves two. If they meet by reference identity (slow == fast), a cycle exists. If fast or fast.next becomes null, there is no cycle.

```
public class DetectCycle {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static boolean hasCycle(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }

    public static void main(String[] args) {
        Node a = new Node(1), b = new Node(2), c = new Node(3);
```

```
        a.next=b; b.next=c; c.next=b;
        System.out.println(hasCycle(a)); // true
    }
}
```

Time O(n), space O(1). Do not compare node values; two different nodes may contain the same value.

12. Find cycle starting node

After slow and fast meet, reset one pointer to head. Move both one step at a time. Their next meeting point is the cycle entry.

```
public class CycleEntry {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node detectCycleStart(Node head) {
        Node slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                Node p = head;
                while (p != slow) {
                    p = p.next;
                    slow = slow.next;
                }
                return p;
            }
        }
        return null;
    }

    public static void main(String[] args) {
        Node a=new Node(1), b=new Node(2), c=new Node(3), d=new Node(4);
        a.next=b; b.next=c; c.next=d; d.next=b;
        System.out.println(detectCycleStart(a).val); // 2
    }
}
```

13. Remove Nth node from end

Use a dummy node before head. Move fast n steps, then move fast and slow together until fast.next is null. slow.next is the node to remove. Dummy makes head deletion easy.

```
public class RemoveNthFromEnd {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node removeNth(Node head, int n) {
        if (n <= 0) throw new IllegalArgumentException();
        Node dummy = new Node(0);
        dummy.next = head;
        Node fast = dummy, slow = dummy;

        for (int i=0; i<n; i++) {
            if (fast.next == null) throw new IllegalArgumentException("n too large");
            fast = fast.next;
        }
        while (fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h=new Node(1); h.next=new Node(2); h.next.next=new Node(3);
        h.next.next.next=new Node(4); h.next.next.next.next=new Node(5);
        print(removeNth(h,2)); // 1 2 3 5
    }
}
```

Time O(n), space O(1).

14. Merge two sorted linked lists

Use a dummy node and tail. Repeatedly attach the smaller current node. When one list ends, attach the remaining suffix. This relinks existing nodes.

```
public class MergeTwoSortedLists {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node merge(Node a, Node b) {
        Node dummy = new Node(0), tail = dummy;
        while (a != null && b != null) {
            if (a.val <= b.val) {
                tail.next = a;
                a = a.next;
            } else {
                tail.next = b;
                b = b.next;
```

```
            }
            tail = tail.next;
        }
        tail.next = (a != null) ? a : b;
        return dummy.next;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node a=new Node(1); a.next=new Node(3); a.next.next=new Node(5);
        Node b=new Node(2); b.next=new Node(4); b.next.next=new Node(6);
        print(merge(a,b)); // 1 2 3 4 5 6
    }
}
```

Time O(m+n), auxiliary space O(1) when relinking original nodes.

15. Remove duplicates from sorted list

Sorted order makes equal values adjacent. If current and current.next match, bypass current.next; otherwise advance current.

```
public class RemoveDuplicatesSorted {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node removeDuplicates(Node head) {
        Node cur = head;
        while (cur != null && cur.next != null) {
            if (cur.val == cur.next.val) cur.next = cur.next.next;
            else cur = cur.next;
        }
        return head;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h=new Node(1); h.next=new Node(1); h.next.next=new Node(2);
        h.next.next.next=new Node(3); h.next.next.next.next=new Node(3);
        print(removeDuplicates(h)); // 1 2 3
    }
}
```

Time O(n), space O(1). For unsorted input, a HashSet can track seen values: expected O(n) time and O(n) space.

16. Palindrome linked list

Find the end of the first half, reverse the second half, compare node values, then restore the second half if the caller expects the input unchanged.

```
public class PalindromeLinkedList {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node reverse(Node h) {
        Node prev=null, cur=h;
        while (cur != null) {
            Node next=cur.next;
            cur.next=prev;
            prev=cur;
            cur=next;
        }
        return prev;
    }

    static boolean isPalindrome(Node head) {
        if (head == null || head.next == null) return true;
        Node slow=head, fast=head;
        while (fast.next != null && fast.next.next != null) {
            slow=slow.next;
            fast=fast.next.next;
        }
        Node second=reverse(slow.next);
        Node p=head, q=second;
        boolean ok=true;
        while (q != null) {
            if (p.val != q.val) { ok=false; break; }
            p=p.next; q=q.next;
        }
        slow.next=reverse(second); // restore
        return ok;
    }

    public static void main(String[] args) {
        Node h=new Node(1); h.next=new Node(2);
        h.next.next=new Node(2); h.next.next.next=new Node(1);
        System.out.println(isPalindrome(h)); // true
    }
}
```

Time O(n), extra space O(1).

17. Intersection of two linked lists

Intersection means the same node object is shared, not merely equal values. Pointer switching makes both pointers travel the same combined distance.

```
public class IntersectionOfLists {
    static class Node {
        int val; Node next;
        Node(int v) { val = v; }
    }

    static Node intersection(Node a, Node b) {
        Node p=a, q=b;
        while (p != q) {
            p = (p == null) ? b : p.next;
            q = (q == null) ? a : q.next;
        }
        return p; // shared node or null
    }

    public static void main(String[] args) {
        Node common=new Node(8); common.next=new Node(10);
        Node a=new Node(1); a.next=new Node(3); a.next.next=common;
        Node b=new Node(2); b.next=common;
        System.out.println(intersection(a,b).val); // 8
    }
}
```

Time O(m+n), extra space O(1). Compare references using ==.

18. Add two numbers represented by lists

Digits are stored in reverse order: 2 -> 4 -> 3 means 342. Add corresponding digits and carry, just like elementary addition.

```
public class AddTwoNumbers {
    static class Node {
        int val; Node next;
        Node(int v) { val=v; }
    }

    static Node add(Node a, Node b) {
        Node dummy=new Node(0), tail=dummy;
        int carry=0;
        while (a != null || b != null || carry != 0) {
            int sum=carry;
            if (a != null) { sum+=a.val; a=a.next; }
            if (b != null) { sum+=b.val; b=b.next; }
            carry=sum/10;
            tail.next=new Node(sum%10);
            tail=tail.next;
        }
        return dummy.next;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node a=new Node(2); a.next=new Node(4); a.next.next=new Node(3);
        Node b=new Node(5); b.next=new Node(6); b.next.next=new Node(4);
        print(add(a,b)); // 7 0 8 means 807
    }
}
```

Time O(max(m,n)), output space O(max(m,n)).

19. Rotate list right by k

Find length and tail. Reduce k with k %= n. Temporarily connect tail to head, move n-k links to find the new tail, then break the ring.

```
public class RotateList {
    static class Node {
        int val; Node next;
        Node(int v) { val=v; }
    }

    static Node rotateRight(Node head, int k) {
        if (head == null || head.next == null || k == 0) return head;
        int n=1;
        Node tail=head;
        while (tail.next != null) { tail=tail.next; n++; }
        k %= n;
        if (k == 0) return head;

        tail.next=head;
        int steps=n-k;
        Node newTail=tail;
        while (steps-- > 0) newTail=newTail.next;
        Node newHead=newTail.next;
        newTail.next=null;
        return newHead;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h=new Node(1); h.next=new Node(2); h.next.next=new Node(3);
        h.next.next.next=new Node(4); h.next.next.next.next=new Node(5);
        print(rotateRight(h,2)); // 4 5 1 2 3
    }
```

```
}
```

Time O(n), extra space O(1). Formula: k = k % n; new tail is at zero-based index n-k-1.

20. Reverse nodes in groups of k

Reverse only complete groups of k. If fewer than k nodes remain, leave them unchanged. groupPrev marks the node before the current group; groupNext marks the node after it.

```
public class ReverseKGroup {
    static class Node {
        int val; Node next;
        Node(int v) { val=v; }
    }

    static Node reverseKGroup(Node head, int k) {
        if (k <= 1 || head == null) return head;
        Node dummy=new Node(0);
        dummy.next=head;
        Node groupPrev=dummy;

        while (true) {
            Node kth=groupPrev;
            for (int i=0; i<k && kth!=null; i++) kth=kth.next;
            if (kth==null) break;

            Node groupNext=kth.next;
            Node prev=groupNext;
            Node cur=groupPrev.next;
            while (cur != groupNext) {
                Node next=cur.next;
                cur.next=prev;
                prev=cur;
                cur=next;
            }
            Node oldHead=groupPrev.next;
            groupPrev.next=kth;
            groupPrev=oldHead;
        }
        return dummy.next;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h=new Node(1); h.next=new Node(2); h.next.next=new Node(3);
        h.next.next.next=new Node(4); h.next.next.next.next=new Node(5);
        print(reverseKGroup(h,2)); // 2 1 4 3 5
    }
}
```

Time O(n), extra space O(1) for this iterative version.

21. Sort linked list using merge sort

Linked lists do not support fast random access, so merge sort is a natural fit. Split with slow/fast pointers, recursively sort each half, then merge. Time O(n log n).

```
public class SortLinkedList {
    static class Node {
        int val; Node next;
        Node(int v) { val=v; }
    }

    static Node sortList(Node head) {
        if (head == null || head.next == null) return head;
        Node slow=head, fast=head.next;
        while (fast != null && fast.next != null) {
            slow=slow.next;
            fast=fast.next.next;
        }
        Node right=slow.next;
        slow.next=null;
        return merge(sortList(head), sortList(right));
    }

    static Node merge(Node a, Node b) {
        Node dummy=new Node(0), tail=dummy;
        while (a != null && b != null) {
            if (a.val <= b.val) { tail.next=a; a=a.next; }
            else { tail.next=b; b=b.next; }
            tail=tail.next;
        }
        tail.next=(a != null) ? a : b;
        return dummy.next;
    }

    static void print(Node h) {
        while (h != null) { System.out.print(h.val+" "); h=h.next; }
    }

    public static void main(String[] args) {
        Node h=new Node(4); h.next=new Node(2);
        h.next.next=new Node(1); h.next.next.next=new Node(3);
        print(sortList(h)); // 1 2 3 4
    }
}
```

Time O(n log n), recursion stack O(log n) for balanced splitting.

22. Doubly linked list — complete implementation

Insertion between left and right requires: node.prev=left; node.next=right; left.next=node; and if right exists, right.prev=node. Head and tail must be updated at boundaries.

```
public class DoublyLinkedList {
    static class Node {
        int val;
        Node prev, next;
        Node(int v) { val=v; }
    }

    private Node head, tail;
    private int size;

    public void addFirst(int x) {
        Node n=new Node(x);
        n.next=head;
        if (head != null) head.prev=n;
        else tail=n;
        head=n;
        size++;
    }

    public void addLast(int x) {
        Node n=new Node(x);
        n.prev=tail;
        if (tail != null) tail.next=n;
        else head=n;
        tail=n;
        size++;
    }

    public int removeFirst() {
        if (head==null) throw new java.util.NoSuchElementException();
        int x=head.val;
        head=head.next;
        if (head != null) head.prev=null;
        else tail=null;
        size--;
        return x;
    }

    public int removeLast() {
        if (tail==null) throw new java.util.NoSuchElementException();
        int x=tail.val;
        tail=tail.prev;
        if (tail != null) tail.next=null;
        else head=null;
        size--;
        return x;
    }

    public void printForward() {
        for (Node c=head; c!=null; c=c.next) System.out.print(c.val+" ");
        System.out.println();
    }

    public void printBackward() {
        for (Node c=tail; c!=null; c=c.prev) System.out.print(c.val+" ");
        System.out.println();
    }

    public static void main(String[] args) {
        DoublyLinkedList d=new DoublyLinkedList();
        d.addFirst(2); d.addFirst(1); d.addLast(3);
        d.printForward();  // 1 2 3
        d.printBackward(); // 3 2 1
        d.removeFirst(); d.removeLast();
        d.printForward();  // 2
    }
}
```

addFirst/addLast/removeFirst/removeLast are O(1). Search by value is O(n).

23. Circular singly linked list — complete implementation

This implementation stores tail; head is tail.next. Empty means tail == null. In a non-empty list, tail.next always points to head.

```
public class CircularSinglyLinkedList {
    static class Node {
        int val; Node next;
        Node(int v) { val=v; }
    }

    private Node tail;
    private int size;

    public void addFirst(int x) {
        Node n=new Node(x);
        if (tail==null) {
            tail=n;
            n.next=n;
        } else {
            n.next=tail.next;
            tail.next=n;
        }
        size++;
    }

    public void addLast(int x) {
        addFirst(x);
```

```
        tail=tail.next;
    }

    public int removeFirst() {
        if (tail==null) throw new java.util.NoSuchElementException();
        Node first=tail.next;
        int x=first.val;
        if (first==tail) tail=null;
        else tail.next=first.next;
        size--;
        return x;
    }

    public void display() {
        if (tail==null) { System.out.println("empty"); return; }
        Node cur=tail.next;
        do {
            System.out.print(cur.val+" ");
            cur=cur.next;
        } while (cur != tail.next);
        System.out.println();
    }

    public static void main(String[] args) {
        CircularSinglyLinkedList c=new CircularSinglyLinkedList();
        c.addLast(10); c.addLast(20); c.addFirst(5);
        c.display(); // 5 10 20
        c.removeFirst();
        c.display(); // 10 20
    }
}
```

24. Circular doubly linked list — pointer rules

Each node has prev and next. For a non-empty list: head.prev == tail and tail.next == head. For a single node, node.next == node and node.prev == node.

```
Insertion at end (non-empty):
new.prev = tail
new.next = head
tail.next = new
head.prev = new
tail = new

Deletion of only node:
head = null
tail = null

Deletion of a known node x (more than one node):
x.prev.next = x.next
x.next.prev = x.prev
if (x == head) head = x.next
if (x == tail) tail = x.prev
```

Always handle the empty and single-node cases separately. A circular list traversal must have a stopping condition based on returning to the start node.

25. Pointer patterns and formulas to memorize

• **Reverse:** next = cur.next; cur.next = prev; prev = cur; cur = next.

• **Middle:** slow advances 1; fast advances 2.

• **Cycle:** slow == fast means cycle; use reference identity.

• **Cycle entry:** after meeting, reset one pointer to head; advance both one step.

• **Nth from end:** advance fast n nodes; then move both until fast.next == null.

• **Merge sorted:** attach smaller current node; then attach remaining suffix.

• **Palindrome:** find first-half end; reverse second half; compare; restore if needed.

• **Intersection:** pointer A walks A then B; pointer B walks B then A.

• **Rotate right:** k %= n; new tail index = n-k-1; new head = newTail.next.

• **Dummy node:** simplifies operations that may change the head.

• **Tail invariant:** in ordinary singly list, tail.next == null.

• **Circular invariant:** tail.next == head.

26. Common edge cases to test

• Empty list

• One node

• Two nodes

• Odd length and even length

• Head is deleted or replaced

• Tail is deleted or replaced

• Duplicate values

• Negative and zero values

• Cycle present

• Invalid index or n greater than list length

• Two lists share a node object

• k = 0, k = 1, k larger than list length

27. Global interview problem bank

Basic operations

• Implement singly linked list

• Implement doubly linked list

• Implement circular linked list

• Insert at head/tail/index

• Delete at head/tail/index

• Search and count nodes

• Find kth node from start

• Find kth node from end

• Delete node when only that node is given (not tail)

Reversal and rearrangement

• Reverse Linked List (iterative and recursive)

• Reverse Linked List II (between positions)

• Reverse Nodes in k-Group

• Swap Nodes in Pairs

• Rotate List

• Reorder List

• Odd-Even Linked List

• Partition List

• Remove Nth Node From End

• Delete Middle Node

Fast/slow and cycle

• Middle of the Linked List

• Linked List Cycle

• Linked List Cycle II

• Palindrome Linked List

• Find duplicate number using cycle-detection idea

Merge, sort and arithmetic

• Merge Two Sorted Lists

• Merge K Sorted Lists

• Sort List (merge sort)

• Add Two Numbers

• Add Two Numbers II (forward digit order)

• Remove Duplicates from Sorted List I and II

Intersection and design

• Intersection of Two Linked Lists

• Copy List with Random Pointer

• LRU Cache

• LFU Cache

• Design Browser History

• Flatten a Multilevel Doubly Linked List

• Design Linked List

• Implement Stack using linked nodes

• Implement Queue using linked nodes

Advanced

• Reverse alternate k nodes

• Josephus problem

• Split circular list into halves

• Merge in Between Linked Lists

• Maximum Twin Sum of a Linked List

• Critical Points in a Linked List

• Remove Zero Sum Consecutive Nodes

• Convert Sorted List to Balanced BST

• Clone list with random pointers

• Rearrange nodes in alternating low/high order

28. Complexity reference

Algorithm	Time	Extra space
Traverse / search / count	O(n)	O(1)
Insert/delete head	O(1)	O(1)
Append with tail pointer	O(1)	O(1)
Reverse iterative	O(n)	O(1)
Reverse recursive	O(n)	O(n) stack
Middle / cycle / cycle entry	O(n)	O(1)
Nth from end	O(n)	O(1)
Merge two sorted lists	O(m+n)	O(1) relinking
Palindrome by reversing half	O(n)	O(1)
Merge sort	O(n log n)	O(log n) stack
Reverse k groups	O(n)	O(1) iterative

29. How to present a linked-list solution in an interview

• Clarify whether the list is singly, doubly, circular, sorted, or may contain cycles.

• Define the node fields and identify head/tail references.

• Draw a small example and state the desired output.

• Name pointer roles: prev, cur, next, slow, fast, dummy, tail.

• State the invariant that must remain true after every update.

• Handle empty, one-node, head, tail and even/odd cases.

• Explain whether you mutate the input list or restore it.

• Give time complexity and auxiliary-space complexity separately.

• Dry-run one normal case and one edge case.

30. Mastery checklist

■ I can implement singly, doubly and circular linked lists from scratch.

■ I can insert and delete at head, tail and index without losing links.

■ I can reverse iteratively and recursively and explain the saved-next rule.

■ I can use slow/fast pointers for middle, cycle and cycle entry.

■ I can remove the nth node from end using a dummy node.

■ I can merge sorted lists and sort a linked list with merge sort.

■ I can solve palindrome, intersection, rotation and reverse-k-group.

■ I can explain pointer invariants and all important edge cases.

■ I can state complexity and compare linked lists with arrays/ArrayList.

Recommended study order: Sections 1–9 first; then middle, cycle, Nth-from-end and merge; then palindrome, intersection, rotate and reverse-k-group; finally doubly/circular lists and design questions.